TAJANI Mohammed Taha

2025/2026

Algorithmique avancée

TP sur les arbres rouge noir

Table des matières

I. Introduction....2
II. Les Arbres Binaires de Recherche....2
II.1) Les méthodes....3
II.2) Les tests....3
III. Les Arbre Rouge Noire....8
III.1) Les méthodes....9
III.2) Rotation Gauche :....12
III.3) Rotation Droite....13
IV. Les tests....15
V. Comparaison : ABR vs ARN....19
VI. Conclusion....24

I. Introduction

L'informatique moderne repose sur le traitement de volumes de données de plus en plus massifs. La rapidité d'accès à l'information est devenue un enjeu critique. Si les processeurs sont devenus très rapides, la manière dont le logiciel organise les données en mémoire reste le facteur limitant principal.

Les structures de données linéaires classiques, comme les tableaux ou les listes chaînées, montrent rapidement leurs limites. Par exemple, rechercher un élément spécifique dans une liste non triée de 1 million d'éléments nécessite, dans le pire des cas, de parcourir toute la liste. Cette complexité linéaire, notée *O(n)*, est inacceptable pour des systèmes performants comme les bases de données, routeurs réseaux et systèmes d'exploitation.

C'est pour répondre à ce besoin d'efficacité que les structures arborescentes ont été développées. L'objectif de ce projet est d'étudier, d'implémenter et de comparer deux structures fondamentales :

- **L'Arbre Binaire de Recherche (ABR)**.
- **L'Arbre Rouge-Noir (ARN)**.

II. Les Arbres Binaires de Recherche

La structure d'Arbre Binaire de Recherche est concrétisée par la classe **ABR** qui hérite de la classe Java **AbstractCollection<T>**. Cette implémentation repose sur une classe interne pour gérer les nœuds contenant les données, ainsi que sur un comparateur pour définir l'ordre de tri, et enfin sur un ensemble de méthodes définissant le comportement standard d'un arbre de recherche.

Chaque élément de l'arbre est matérialisé par la classe interne **NoeudArbre**. Cette classe regroupe les attributs essentiels : la valeur stockée, les références vers les enfants (**filsGauche** et **filsDroit**) ainsi qu'un lien vers le **noeudParent**.

La classe **NoeudArbre** intègre également deux méthodes utilitaires : **obtenirMinimum()**, qui permet de descendre jusqu'à la plus petite valeur d'un sous-arbre, et **obtenirSuccesseur()**, qui identifie le nœud suivant dans l'ordre croissant.Concernant l'initialisation, la classe **ABR** propose trois constructeurs distincts :

- **ABR()** : Crée un arbre vide utilisant l'ordre naturel des éléments.
- **ABR(Comparator<? super T> comp)** : Instancie un arbre utilisant un comparateur spécifique fourni en paramètre.

- **ABR(Collection<? extends T> collection)** : Construit l'arbre en y insérant directement l'ensemble des éléments présents dans la collection donnée.

II.1) Les méthodes

Dans cette partie, nous décrivons brièvement les fonctionnalités clés que nous avons implémentées dans notre classe ABR :

- **size()** : Cette méthode retourne simplement la taille actuelle de l'arbre.
- **insererTous(Collection<? extends T> collection)** : Cette méthode est conçue pour ajouter efficacement toute une liste d'éléments dans l'arbre. Elle utilise une boucle pour parcourir la collection et appelle la méthode **add** pour chaque élément, garantissant ainsi qu'ils sont tous insérés un par un. Elle renvoie **true** si l'arbre a été modifié.
- **contains()** : Elle permet de vérifier la présence d'un élément. Elle s'appuie sur la méthode privée **trouverNoeud**, qui descend dans l'arbre en comparant la clé cherchée avec les nœuds actuels pour localiser l'élément.
- **remove()** : Cette méthode permet de retirer un élément de l'arbre. Elle fait appel à une méthode privée complexe, **supprimerNoeud**, qui se charge de détacher le nœud visé et de réorganiser les liens entre les parents et les enfants restants pour garder l'arbre cohérent.
- **add()** : C'est la méthode d'insertion. Elle place un nouvel élément au bon endroit en respectant l'ordre défini par le comparateur. Elle descend jusqu'à trouver une position libre (feuille null) pour y accrocher le nouveau **NoeudArbre**.
- **La classe interne IterateurArbre** : Nous avons développé cet itérateur pour pouvoir parcourir les éléments de l'arbre un par un. Il implémente les méthodes standards : **hasNext()** pour vérifier s'il reste des éléments, **next()** pour récupérer l'élément suivant, et **remove()** pour supprimer l'élément en cours de visite.

II.2) Les tests

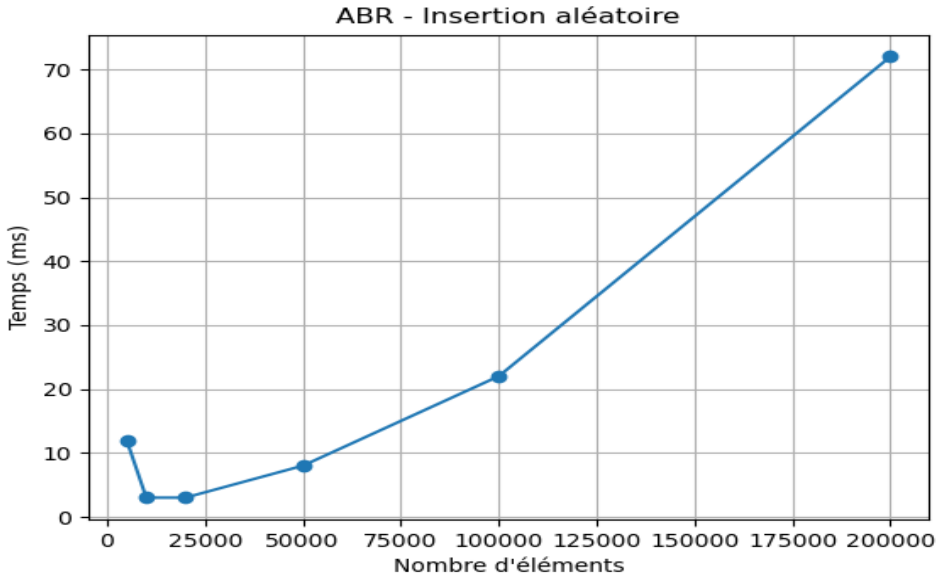
Dans cette section, nous évaluons l'efficacité de ABR en mesurant les temps d'exécution pour l'insertion et la recherche. Nous distinguons deux scénarios : le cas moyen (insertion aléatoire) et le cas défavorable (insertion ordonnée).

- **Temps d’insertion (Cas Aléatoire)**

L'insertion aléatoire consiste à insérer des clés générées sans ordre particulier. Ce scénario simule l'arrivée de données non triées, conduisant généralement à un arbre relativement équilibré.

Taille	Temps en ms
5000	12
10000	3
20000	3
50000	8
100000	22
200000	72

- **Analyse :** On observe une augmentation progressive du temps de construction corrélée à la taille des données. La structure reste efficace pour des volumes modérés. Cependant, à partir de 100000 éléments, et surtout pour 200000 d'éléments, le temps de construction croît de manière plus significative. Cela démontre que bien que l'ABR soit performant, le coût de gestion de la structure augmente avec la profondeur de l'arbre.

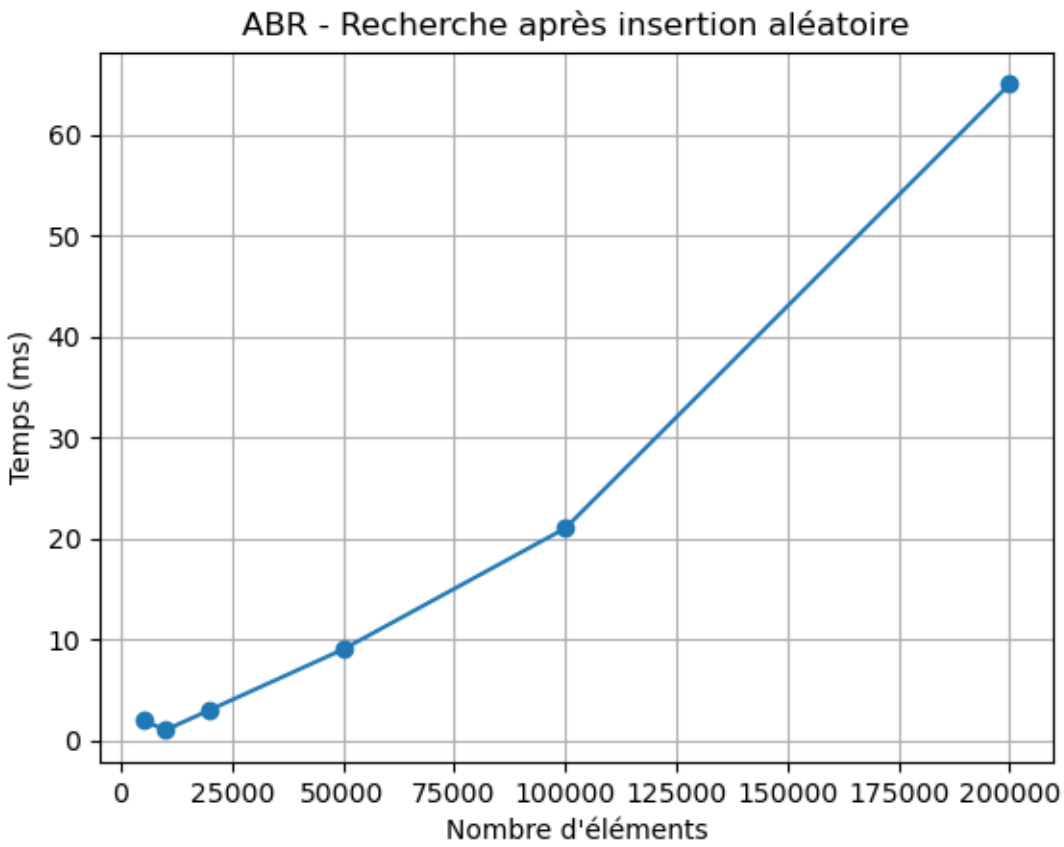


- **Temps de Recherche (Cas Aléatoire)**

Ici pour le temps de recherche, on prend un cas où on teste le temps de recherche de l'arbre avec N clefs.

Taille	Temps en ms
5000	2
10000	1
20000	3
50000	9
100000	21
200000	65

- **Analyse :** L'étude du temps de recherche dans le cas d'insertion aléatoire montre que l'arbre est très performant. Contrairement aux résultats précédents, l'arbre répond instantanément même pour de grandes plages de données. Le temps de recherche moyen est excellent , ce qui confirme que la complexité reste proche de *O(logN)*.

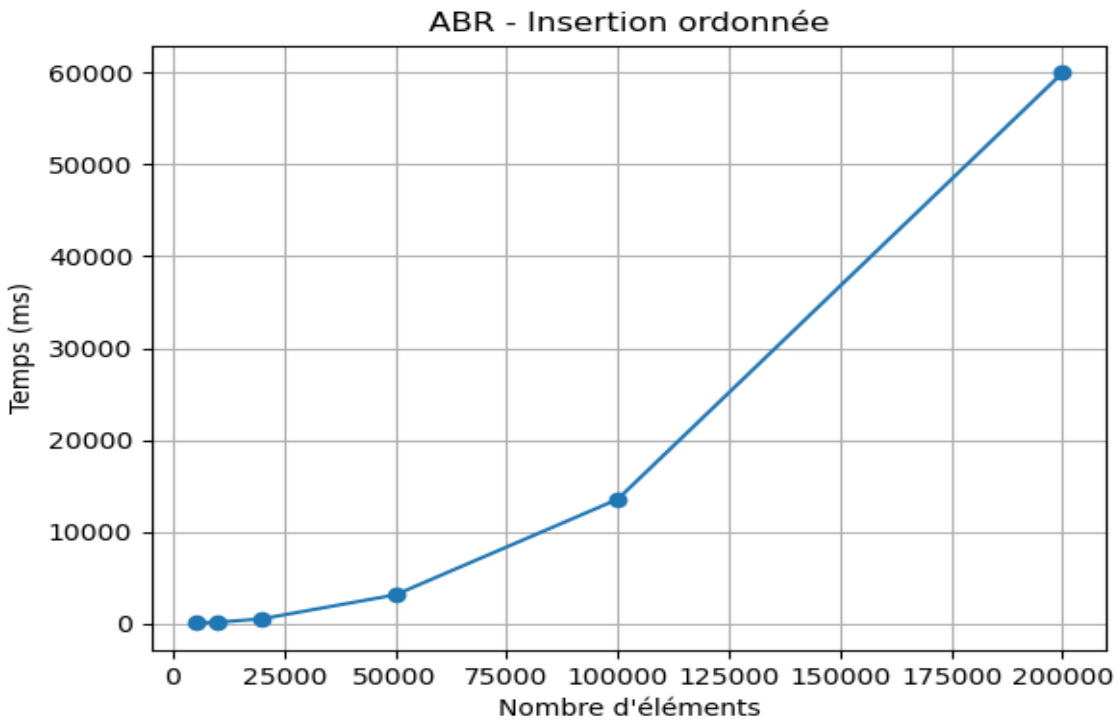


- **Temps d’insertion ordonnée (cas défavorable)**

On va insérer les clés dans un ordre croissant. Cela conduit à un arbre dégénéré où chaque nœud a uniquement un enfant droit, ce qui ressemble à une structure linéaire par exemple liste chaînée.

Taille	Temps en ms
5000	43
10000	134
20000	503
50000	3115
100000	13535
200000	60003

- **Analyse :** On constate une augmentation brutale et exponentielle du temps d’insertion. On passe de quelques millisecondes en aléatoire à **plus de 60 secondes** (60 003 ms) pour 200 000 éléments triés. Au-delà de cette taille, le programme met un temps considérable (voire sature la pile d'appel).



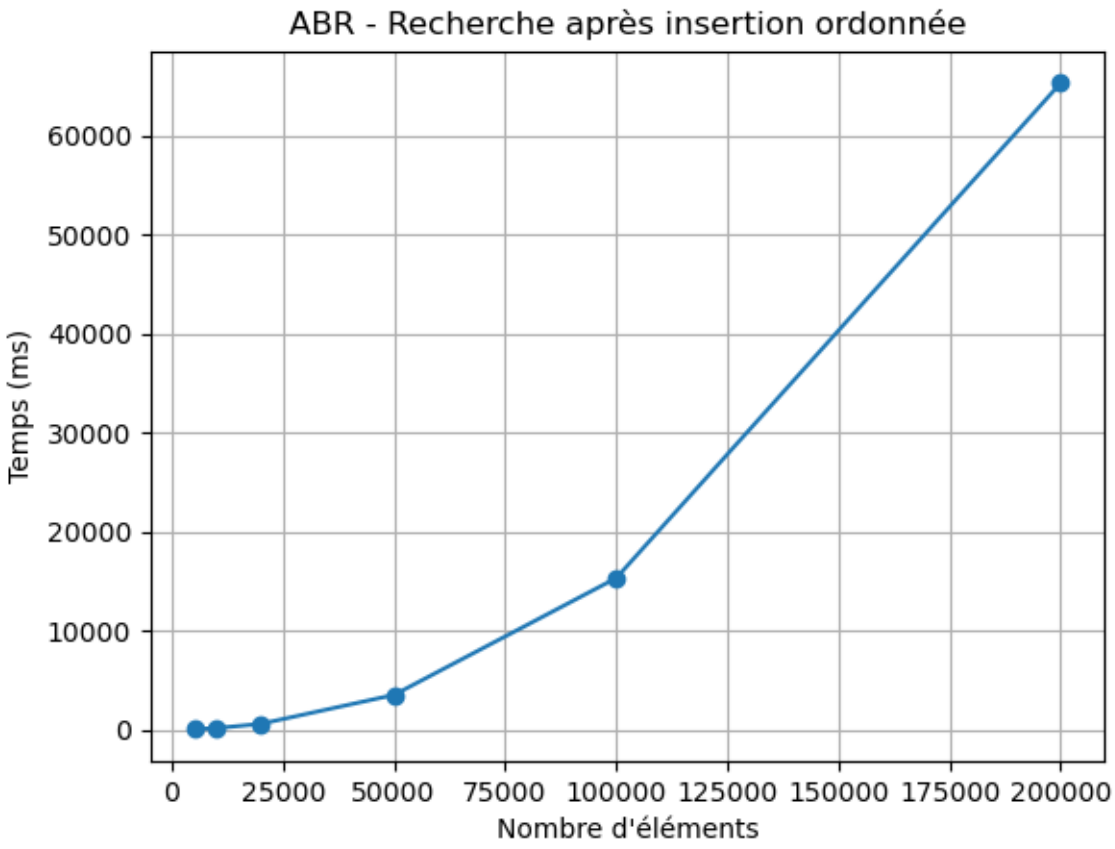
On peut ici conclure d’après les résultats obtenus que l’insertion est de complexité $O(N^2)$ au global ou $O(N)$ par insertion, puisque l’arbre est devenu unilatéral.

- **Temps de recherche ordonné (cas défavorable)**

On mesure le temps de recherche pour le cas défavorable, et on obtient les résultats suivants :

Taille	Temps en ms
5000	32
10000	138
20000	565
50000	3494
100000	15299
200000	65310

- **Analyse :** Le temps de recherche n'est plus "raisonnable" du tout. Il suit la même dégradation que l'insertion. Pour 200 000 éléments, la recherche prend environ **65 secondes**, contre seulement 65 millisecondes dans le cas aléatoire. C'est 1000 fois plus lent. Cela confirme que pour trouver un élément au fond de l'arbre dégénéré, il faut parcourir tous les nœuds précédents.



III. Les Arbre Rouge Noire

Un arbre rouge-noir est une variante d'arbre binaire de recherche dotée de propriétés additionnelles garantissant que l'arbre reste approximativement équilibré après chaque insertion ou suppression. Cette structure permet de garantir une complexité logarithmique *O(logN)* pour les opérations de base.

Les propriétés fondamentales qui caractérisent un ARN sont :

- **Couleur :** Chaque nœud est soit **rouge**, soit **noir**. La racine est toujours **noire**.
- **Parenté :** Si un nœud est **rouge**, alors ses deux enfants doivent être **noirs**. Autrement dit, il ne peut pas y avoir deux nœuds rouges consécutifs.
- **Équilibre (Hauteur Noire) :** Pour tout nœud x, tous les chemins descendant de x vers ses feuilles doivent contenir le même nombre de nœuds **noirs**.

➢ *La Classe **ARN** :*

Pour implémenter cette structure nous avons conçu une classe générique **ARN<E>** permettant de gérer des éléments de n'importe quel type.

- **Héritage :** Elle hérite de la classe **AbstractCollection<E>**, fournissant ainsi une implémentation partielle de l'interface Collection standard de Java.
- **Attributs principaux :**
  - **racine** : Le point d'entrée de l'arbre.
  - **cmp** : Un **Comparator** pour définir l'ordre des éléments.
  - **nil (Sentinelle)** : Un nœud spécial remplaçant les pointeurs null. Toutes les feuilles de l'arbre pointent vers cette sentinelle, qui est toujours considérée comme noire. Cela simplifie la gestion des cas limites.

➢ *La Classe Interne **Noeud** :*

La structure fondamentale de l'arbre est définie par une classe interne **Noeud**. Chaque instance encapsule :

- La **clé**.
- Les références vers le **parent**, l'enfant **gauche** et l'enfant **droit**.
- La **couleur**.

Méthodes clés de la classe Noeud :

- **Constructeur :** Lors de la création d'un nœud, on lui affecte la clé transmise. La **couleur initiale est fixée à ROUGE**.

- **minimum() :** Retourne l'élément le plus petit du sous-arbre (le nœud le plus à gauche).
- **suivant() :** Trouve le successeur immédiat d'un nœud dans l'ordre trié (utile pour l'itération).
- **equals() :** Permet de vérifier l'égalité entre deux instances de nœuds.

➢ *Constructeurs de l'**ARN :***

- **Constructeur par défaut ARN() :** Initialise un arbre vide. Il crée un comparateur anonyme, instancie la sentinelle *nil*, et définit la racine sur *nil*.
- **Constructeur avec Comparateur ARN(Comparator<? super E> cmp) :** Similaire au précédent, mais permet l'usage d'un comparateur personnalisé. La racine est initialisée à *nil* et l'attribut *cmp* est affecté.
- **Constructeur par Collection ARN(Collection<? extends E> c) :** Permet de construire un arbre directement à partir d'une liste existante. Il appelle le constructeur par défaut , puis parcourt la collection c via une boucle *for-each* pour insérer chaque élément via la méthode **add()**. Cela garantit que l'arbre est ordonné dès sa création.

III.1) Les méthodes

La manipulation des Arbres Rouge Noir repose sur deux opérations fondamentales : l'ajout et la suppression. Contrairement aux ABR classiques, ces opérations nécessitent des étapes de rééquilibrage (rotations et recolorations) pour garantir le respect des invariants.

➢ *La Suppression **(`remove`):***

La suppression est l'opération la plus complexe dans un ARN. Elle se décompose en une méthode publique d'interface et une logique interne de gestion des cas.

➔ Méthode **remove(Object o) :**

Cette méthode sert d'interface avec l'utilisateur. Elle appelle d'abord **rechercher(o)** pour localiser le nœud cible.

Si le nœud n'est pas trouvé la méthode retourne *false*.

Si le nœud est trouvé elle appelle la méthode interne **supprimer(noeud)** et retourne *true*.

➔ Méthode interne **supprimer(Noeud z) :**

Cette méthode retire le nœud z tout en préparant le terrain pour le rééquilibrage.

- **Cas triviaux :** Si z est la sentinelle, on ne fait rien. Si z est la racine unique, l'arbre est vidé.
- **Substitution :** L'algorithme détermine un nœud de remplacement e.
- **Conservation immédiate :** Si le nœud supprimé z ou son remplaçant e était **ROUGE**, la suppression n'altère pas la "hauteur noire" de l'arbre. Le remplacement est simple et aucune correction n'est requise.
- **Le problème du Double Noir:** Si on supprime un nœud **NOIR** et qu'on le remplace par un enfant **NOIR**, un déséquilibre survient. Le nœud remplaçant est alors marqué virtuellement comme "Double Noir". Il faut lancer la procédure de correction.

➔ Correction du Double Noir **(supprimerCorrection) :**

L'objectif est de transférer ce noir en trop vers le haut de l'arbre ou de l'éliminer via des rotations. Soit *ptr* le nœud à corriger, *pere* son parent, et *s* son frère. L'algorithme distingue le cas où ptr est enfant gauche ou enfant droit. Voici le cas où *ptr* est enfant gauche :

- **Cas 1 : Le frère *s* est ROUGE.**
  - On recolorie *s* en noir et *pere* en rouge, puis on effectue une **Rotation Gauche** sur *pere*.
  - *Résultat :* Le frère devient noir, ce qui nous ramène aux cas suivants.
- **Cas 2 : Le frère *s* est NOIR et ses deux enfants sont NOIRS.**
  - On recolorie *s* en rouge.
  - *Résultat :* Le noir en trop remonte vers le parent *pere*. La boucle continue avec *pere* comme nouveau *ptr*.

- **Cas 3 : Le frère *s* est NOIR avec des enfants mixtes.**
    - Si l'enfant gauche de *s* est rouge et le droit noir : On fait une **Rotation Droite** sur *s*. *s* devient rouge, son enfant devient noir. On se ramène au **Cas 4**.
- **Cas 4 :** Si l'enfant droit de *s* est rouge : On recolorie s avec la couleur du *pere*, on met le *pere* en noir, l'enfant droit de *s* en noir, et on effectue une **Rotation Gauche** sur *pere*. Le double noir est éliminé.

➢ *L'Insertion **(add):***

L'ajout d'un élément se fait en deux temps : une insertion classique type ABR, suivie d'une correction des violations de couleur.

➔ Méthode interne **supprimer(Noeud z) :**

- **Validation :** On vérifie que l'élément n'est pas *null*.
- **Création :** On crée un nouveau nœud avec la couleur **ROUGE**.
- **Placement :** On parcourt l'arbre gauche si valeur < clé, droite sinon jusqu'à trouver une feuille. Le nouveau nœud est lié à son parent.
- **Correction :** Si le parent du nouveau nœud est aussi **ROUGE**, on appelle **ajouterCorrection()** pour résoudre le conflit.

➔ Méthode **ajouterCorrection(Noeud z)**

Tant que le parent de z est **ROUGE**, nous avons une violation. La résolution dépend de la couleur de l'oncle *y* du nœud *z*.

- **Cas A : L'oncle *y* est ROUGE.**
    - Recoloration simple. Le parent et l'oncle deviennent **NOIRS**. Le Grand-Parent devient **ROUGE**.
    - *Résultat :* Le problème remonte au niveau du Grand-Parent.
- **Cas B : L'oncle *y* est NOIR .**
    - Il faut effectuer des rotations pour restructurer l'arbre.
    - *Sous-cas Triangle:* Si *z* est enfant droit et le parent enfant gauche, on fait une **Rotation Gauche** sur le parent.

- *Sous-cas Ligne :* Si *z* est enfant gauche et le parent enfant gauche, on recolorie le parent en **NOIR**, le Grand-Parent en **ROUGE**, et on effectue une **Rotation Droite** sur le Grand-Parent.

III.2) Rotation Gauche :

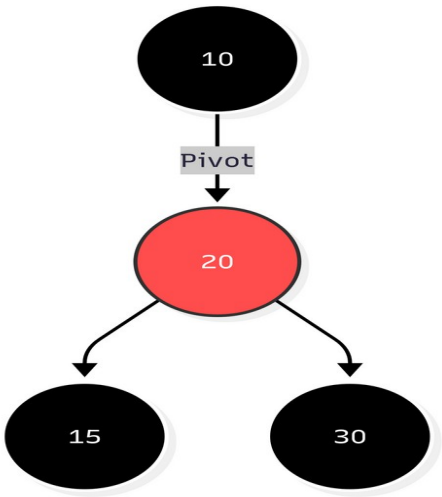
Cette méthode est essentielle dans les ARN pour rééquilibrer l'arbre après une insertion ou une suppression qui aurait violé les propriétés de couleur ou de hauteur. La rotation gauche transforme la structure locale d'un arbre pour déplacer un nœud (le pivot) vers le bas à gauche, tout en remontant son enfant droit.

Algorithme d'implémentation :

- **Déplacement du sous-arbre gauche de l'enfant droit :** Le sous-arbre gauche de l'enfant droit (du pivot) est détaché et devient le sous-arbre droit du nœud pivot.
- **Reconfiguration des parents :** Le parent de l'enfant droit devient le parent du nœud pivot. Si le pivot était la racine de l'arbre, l'enfant droit devient la nouvelle racine.
- **Réassignation des branches :** Le nœud pivot devient l'enfant gauche de son ancien enfant droit.

Exemple d'application : Prenons la configuration suivante :

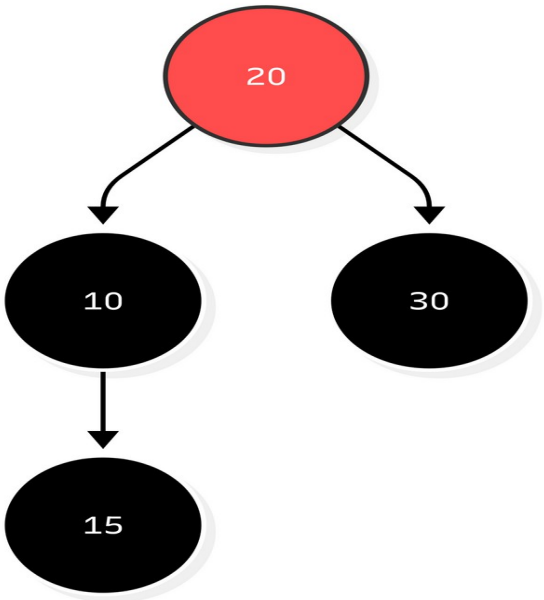
- **10** est le nœud pivot (Noir).
- **20** est l'enfant droit de 10 (Rouge).
- **15** et **30** sont les enfants (Noirs) de 20.



Les étapes de la rotation gauche sur 10 :

- **Déplacement du sous-arbre :** Le nœud **15** (enfant gauche de 20) est déplacé pour devenir l'enfant droit de **10**.
- **Réassignation : 20** remonte et prend la place de 10 (il devient la racine de ce sous-arbre).
- **Liaison finale : 10** descend et devient l'enfant gauche de **20**.

État Après la Rotation Gauche :



III.3) Rotation Droite

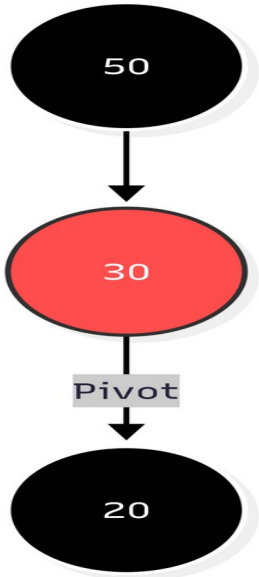
La rotation droite est l'opération symétrique. Elle est utilisée pour rééquilibrer un arbre rouge-noir lorsque l'arbre penche trop à gauche.

Algorithme d'implémentation :

- **Identification du pivot :** Le nœud sur lequel la rotation est effectuée.
- **Réorganisation des sous-arbres :** Le sous-arbre **droit** de l'enfant **gauche** est détaché pour devenir le sous-arbre **gauche** du pivot.
- **Remontée :** L'enfant gauche monte et devient le parent du pivot.
- **Mise à jour racine :** Si le pivot était la racine, l'enfant gauche devient la nouvelle racine.

Exemple d'application : Prenons l'arbre suivant pour montrer le processus :

- **50** est la racine de l'arbre global.
- **30** est l'enfant gauche de 50 (C'est notre **Pivot**).
- **20** est l'enfant gauche de 30.

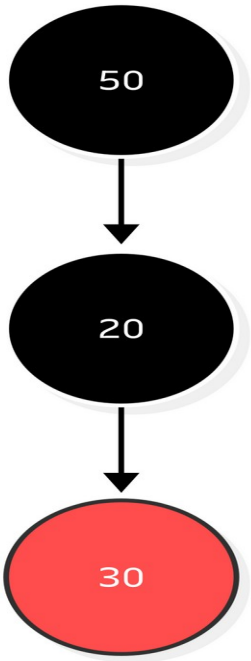


État Avant la Rotation (sur le nœud 30) :

Les étapes de la rotation droite sur 30 :

- **20** monte pour prendre la place de 30.
- **30** descend et devient l'enfant **droit** de 20.
- Le sous-arbre droit de 20 (qui est vide ici) deviendrait le sous-arbre gauche de 30.
- **20** est rattaché au parent supérieur (**50**) comme nouvel enfant gauche.

État Après la Rotation Droite :



III.4) Les tests

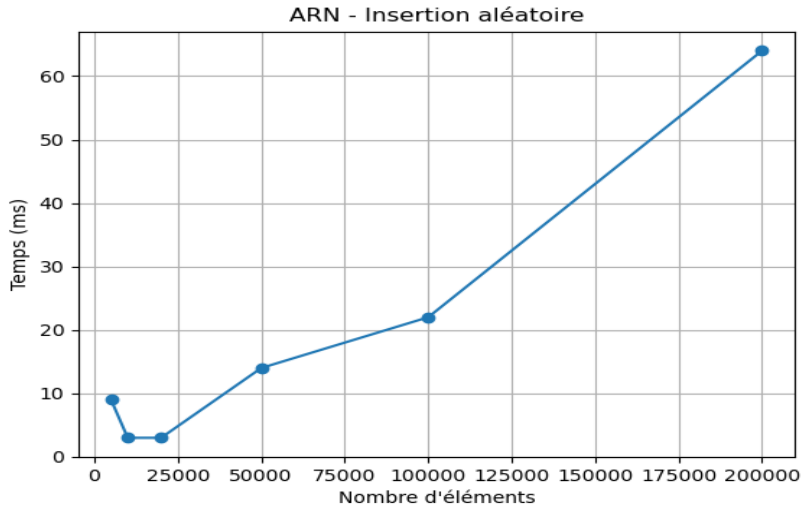
- **Temps d'insertion (Cas Aléatoire)**

Nous débutons par l'analyse de l'insertion avec des données non triées (générées aléatoirement). Dans ce scénario, chaque ajout déclenche potentiellement des mécanismes de correction (rotations et recolorations) afin de préserver les invariants de l'ARN, notamment l'absence de doubles rouges et l'équilibre de la hauteur noire. L'objectif est de vérifier si ce surcoût de traitement impacte la rapidité d'exécution.

Taille	Temps en ms
5000	9
10000	3
20000	3
50000	14
100000	22
200000	64

Analyse : Les mesures confirment l'efficacité de la structure :

- **Performance maintenue :** L'évolution des temps reste minime malgré l'augmentation du volume de données. Pour 200 000 éléments, l'insertion ne requiert que **64 ms**.
- **Coût négligeable :** Contrairement à ce qu'on pourrait craindre, les opérations de rééquilibrage ne ralentissent pas le processus. Les temps sont quasi-identiques, voire légèrement meilleurs, que ceux d'un ABR classique sans équilibrage. La complexité empirique respecte bien la courbe logarithmique *O(logN)*.



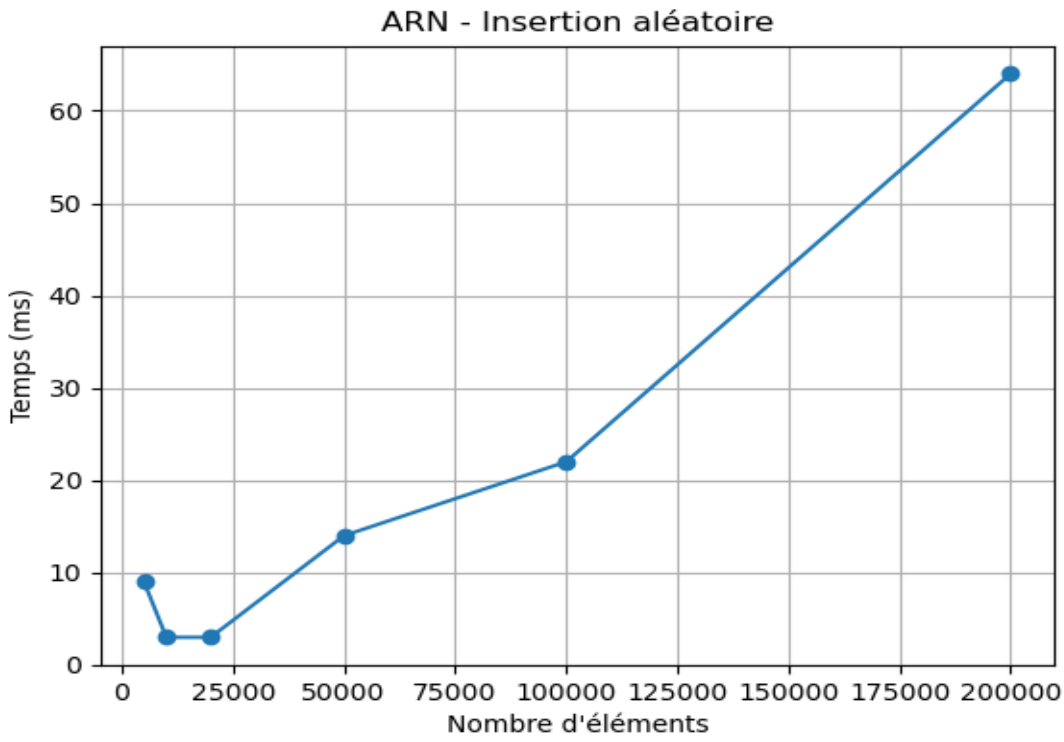
- **Temps de recherche (Cas Aléatoire)**

Nous évaluons ensuite la rapidité d'accès aux données au sein de l'arbre construit précédemment. Ce test mesure la capacité de l'ARN à parcourir ses branches efficacement.

Taille	Temps en ms
5000	5
10000	1
20000	2
50000	7
100000	18
200000	80

Analyse :

- **Stabilité :** Le temps de recherche évolue de manière très maîtrisée. Le passage de 100 000 à 200 000 éléments n'entraîne pas d'explosion du temps de réponse .
- **Hauteur maîtrisée :** Ces résultats attestent que l'arbre est resté compact. La hauteur maximale étant bornée par $2log_2(N+1)$, le nombre de nœuds traversés pour atteindre une cible reste faible.



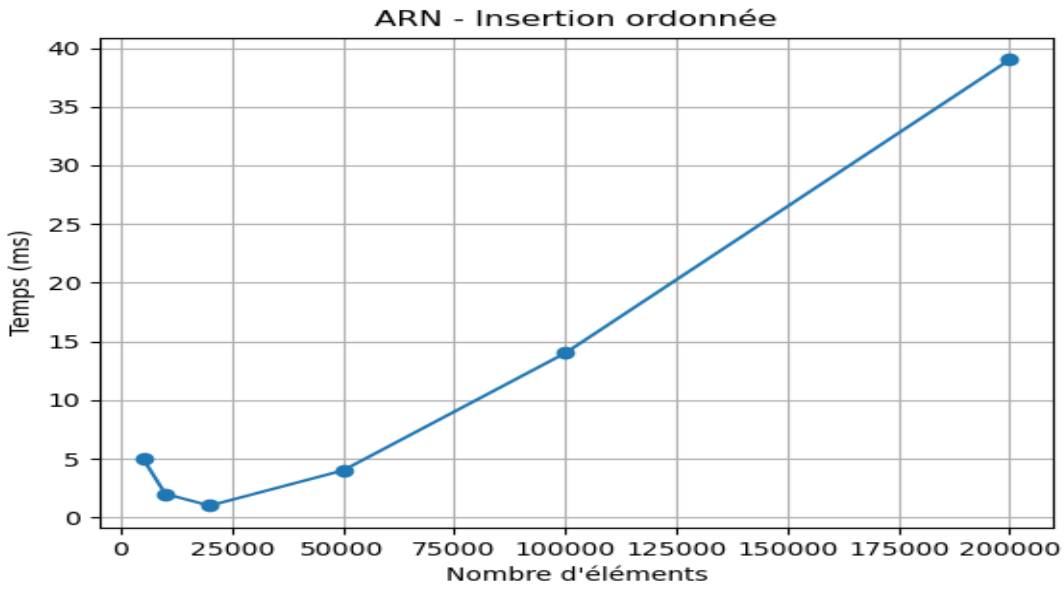
- **Temps d’insertion ordonnée (cas défavorable)**

C'est dans cette configuration que l'Arbre Rouge-Noir démontre sa supériorité structurelle. Nous soumettons l'arbre à une séquence de données triées (ordre croissant), le scénario exact qui avait provoqué l'effondrement des performances de l'ABR standard.

Taille	Temps en ms
5000	5
10000	2
20000	1
50000	4
100000	14
200000	39

Analyse : L'écart de performance est spectaculaire :

- **Résilience totale :** Là où l'ABR nécessitait une minute (**60 003 ms**) pour traiter 200 000 éléments triés, l'ARN accomplit la tâche en seulement **39 ms**.
- **Gain d'efficacité :** Le facteur d'accélération est d'environ **1500x**.
- **Explication technique :** Grâce aux rotations automatiques, l'ARN empêche la formation d'une branche unique. L'arbre reste large et peu profond, garantissant que l'insertion reste en complexité *O(logN)* et non *O(N2)*.

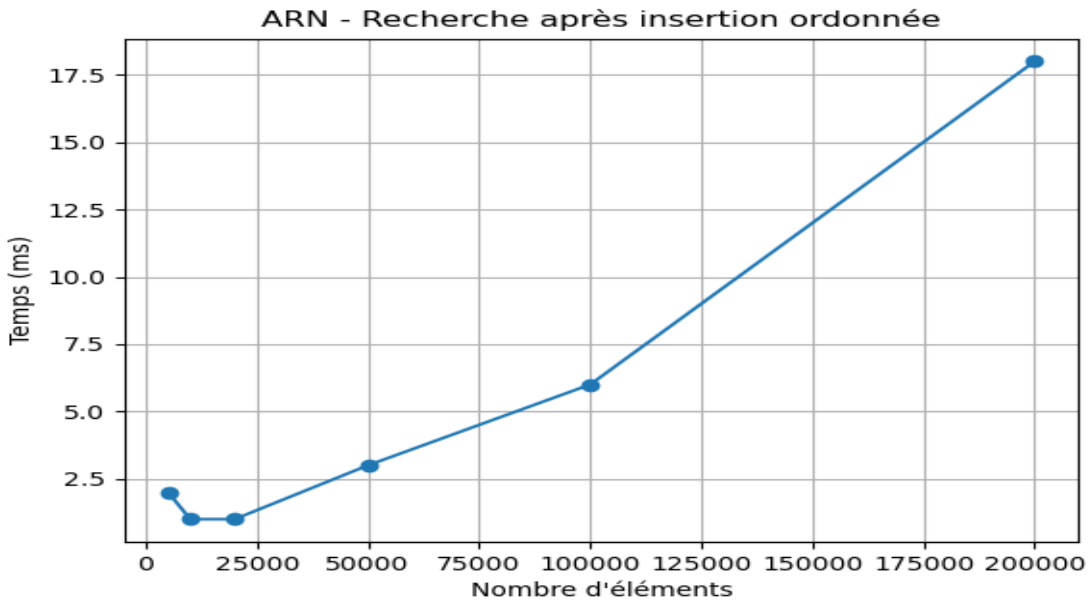


- **Temps de Recherche ordonnée (cas défavorable)**

Taille	Temps en ms
5000	2
10000	1
20000	1
50000	3
100000	6
200000	18

Analyse :

- **Excellence maintenue :** La recherche est instantanée (**18 ms** pour 200 000 tests), ce qui contraste violemment avec les 65 secondes nécessaires pour l'ABR dans le même contexte.
- **Paradoxe de performance :** On note que la recherche dans cet arbre "ordonné" (18 ms) est même plus rapide que dans le cas aléatoire (80 ms). Cela peut s'expliquer par une allocation mémoire plus contiguë ou une topologie d'arbre particulièrement régulière optimisant l'usage du cache processeur.
- **Conclusion :** L'ARN offre une garantie de performance universelle, indépendante de la qualité ou de l'ordre des données d'entrée.



IV. Comparaison : ABR vs ARN

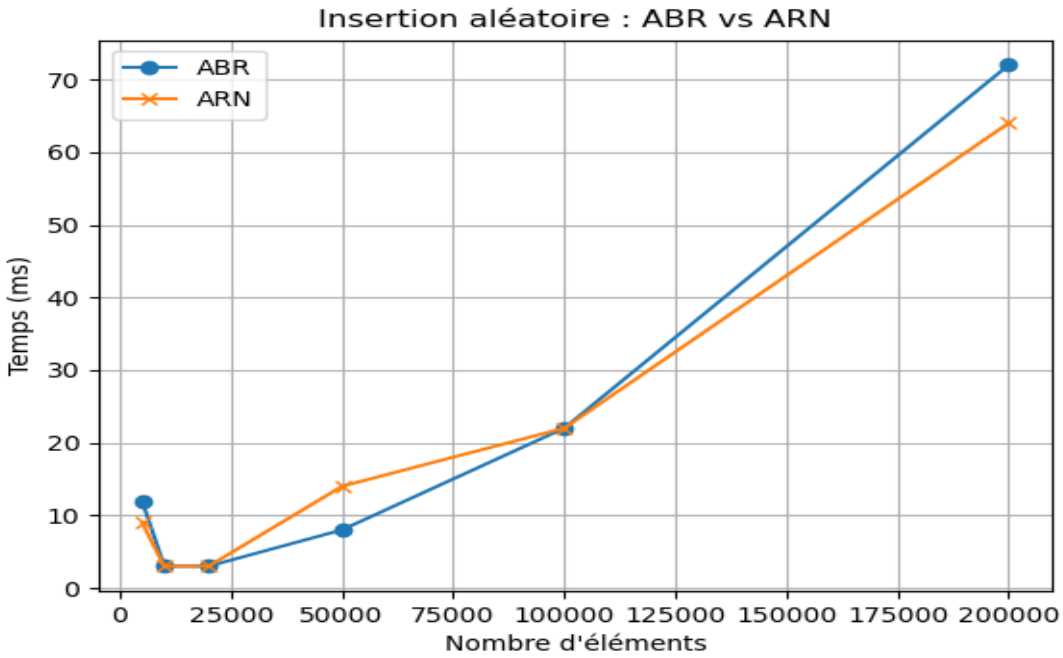
Dans cette section de synthèse, nous confrontons directement les performances des Arbres Binaires de Recherche classiques et des Arbres Rouge Noir. L'objectif est de mettre en évidence les limites de l'ABR et de démontrer comment l'ARN surmonte ces faiblesses, tant pour l'insertion que pour la recherche.

- **L’insertion aléatoire**

Dans le cas d'une insertion aléatoire, les données arrivent de manière désordonnée, ce qui tend naturellement à équilibrer un ABR, bien que le risque de déséquilibre local persiste. L'Arbre Rouge-Noir, quant à lui, applique systématiquement ses mécanismes de correction pour garantir une hauteur minimale, quelle que soit la distribution des clés.

Taille	Temps en ms (ABR)	Temps en ms (ARN)
5000	12	9
10000	3	3
20000	3	3
50000	8	14
100000	22	22
200000	72	64

Analyse de la courbe comparative :



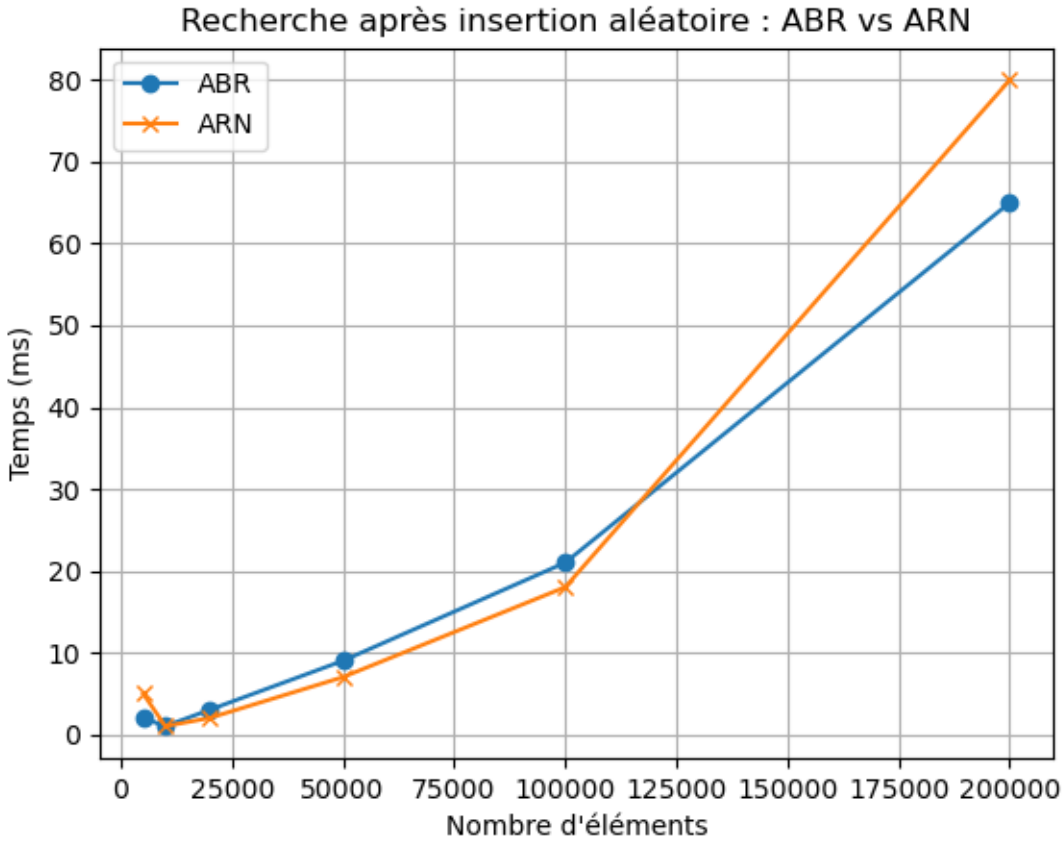
D'après la courbe ci-dessus, nous observons que les deux structures offrent des performances comparables sur de petits volumes. Cependant, l'ARN démontre une stabilité supérieure. Alors que la courbe de l'ABR tend à croître plus nettement à partir de 20 000 éléments, celle de l'ARN reste linéaire et faible. Cette maîtrise de la croissance favorise l'utilisation des Arbres Rouge-Noir pour les grands volumes de données, même aléatoires.

- **Temps de recherche (Cas aléatoire)**

Nous comparons ici la vitesse d'accès aux données. C'est un indicateur critique de la qualité de la structure générée lors de l'insertion.

Taille	Temps en ms (ABR)	Temps en ms (ARN)
5000	2	5
10000	1	1
20000	3	2
50000	9	7
100000	21	18
200000	65	80

Analyse de la courbe comparative :



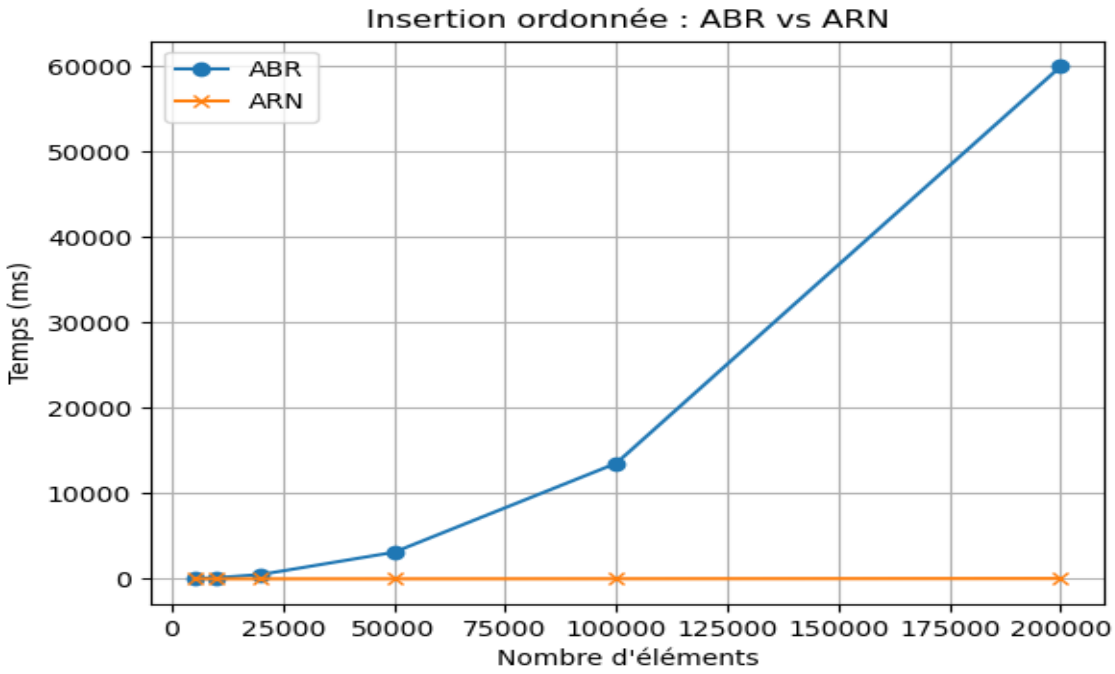
L'analyse graphique révèle une différence notable de comportement. Le temps de recherche dans l'ABR, bien que raisonnable, subit les effets des légers déséquilibres créés lors de l'insertion aléatoire, entraînant une courbe ascendante. À l'inverse, le temps de recherche pour l'ARN est quasi-négligeable (restant souvent sous la barre des 10 à 20 ms, même pour de grandes tailles). La rigueur de l'équilibrage de l'ARN garantit un chemin d'accès optimal vers les nœuds, marquant ici un point décisif en faveur des ARNs.

- **L'insertion ordonnée (Le cas défavorable)**

Ce scénario constitue le crash test de notre étude. Nous insérons les clés dans un ordre strictement croissant. Pour un ABR, c'est le cas défavorable par excellence . Pour l'ARN, c'est un test de sa capacité d'auto-équilibrage.

Taille	Temps en ms (ABR)	Temps en ms (ARN)
5000	43	5
10000	134	2
20000	503	1
50000	3115	4
100000	13535	14
200000	60003	39

Analyse de la courbe comparative :



La confrontation des courbes est sans appel.

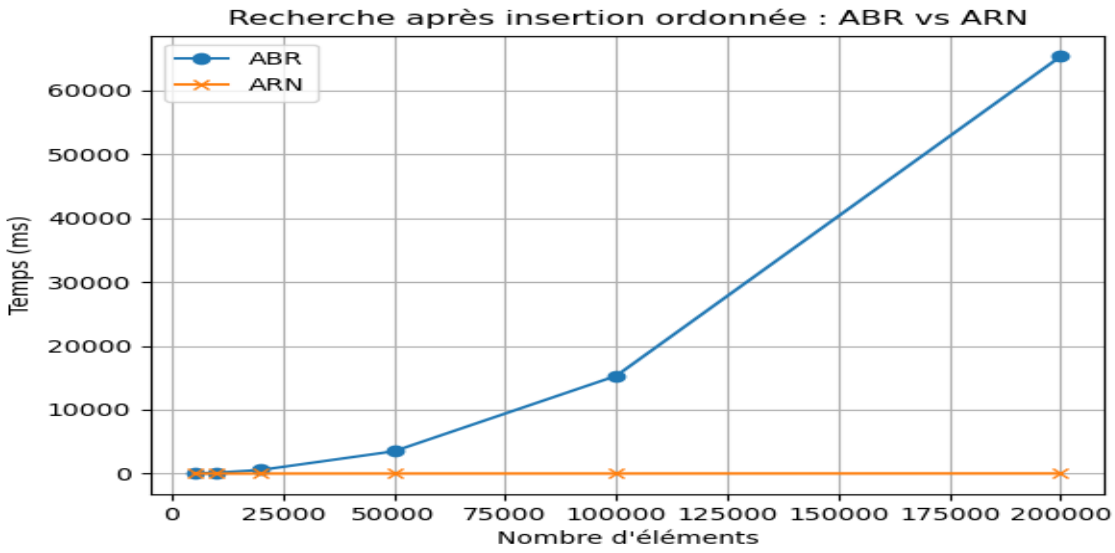
- **Pour l'ABR :** La courbe s'envole verticalement dès 5 000 éléments. Le temps de construction explose, rendant la structure inutilisable.
- **Pour l'ARN :** L'insertion ordonnée n'a pratiquement aucun impact négatif. Grâce aux rotations effectuées à la volée, la courbe reste plate et proche de zéro. L'ARN surclasse totalement l'ABR dans ce scénario, prouvant sa robustesse face aux données pré-triées.

- **Temps de recherche ordonnée(Cas défavorable)**

Enfin, nous testons la recherche dans les arbres construits précédemment. La qualité de la recherche étant tributaire de la hauteur de l'arbre, nous nous attendons à retrouver les écarts constatés lors de l'insertion.

Taille	Temps en ms (ABR)	Temps en ms (ARN)
5000	32	2
10000	138	1
20000	565	1
50000	3494	3
100000	15299	6
200000	65310	18

Analyse de la courbe comparative :



Comme le montre la courbe, le temps de recherche pour l'ARN reste extrêmement faible et constant, confirmant que l'arbre est resté parfaitement équilibré malgré l'insertion séquentielle. À l'opposé, la courbe de l'ABR augmente brutalement et linéairement avec la taille des données. Chercher dans cet ABR revient à parcourir une liste chaînée *O(N)*. Cette divergence illustre définitivement la supériorité de l'Arbre Rouge-Noir pour garantir des temps de réponse rapides en toutes circonstances.

V. Conclusion

Cette étude comparative met en lumière la différence fondamentale entre les deux structures : la **gestion du déséquilibre**.

- **Arbres Binaires de Recherche (ABR) :** Ils sont simples à implémenter et performants tant que les données sont aléatoires. Cependant, ils sont instables : dans le pire des cas, leurs performances s'effondrent pour atteindre une complexité linéaire *O(N)*, tant en insertion qu'en recherche.
- **Arbres Rouge-Noir (ARN) :** Grâce à leurs invariants et aux mécanismes de rotations, ils garantissent une complexité logarithmique *O(logN)* dans **tous les cas**.

En conclusion, bien que l'ABR puisse suffire pour des cas simples, l'Arbre Rouge-Noir s'impose comme la solution indispensable pour des applications industrielles nécessitant fiabilité, stabilité et performance à grande échelle.